



University of Minho
School of Engineering



cesium



BeLIUM Viagens

Do Minho para o Mundo

Base de Dados
Licenciatura em Engenharia Informática 2025/26

EQUIPA DE TRABALHO

Grupo 55

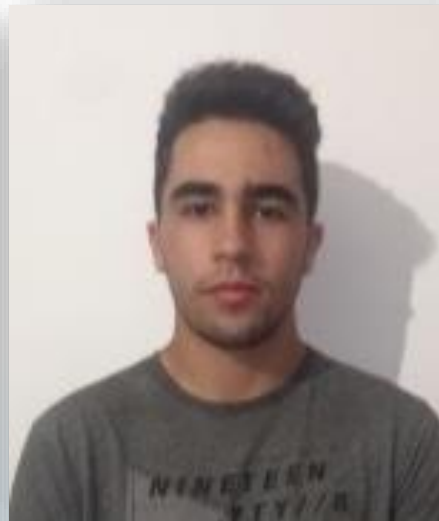
Diogo Azevedo
109727



Vera Almeida
110723



Bruno Magalhães
110274



Simão Santos
109767



ESTRUTURA DA APRESENTAÇÃO

Definição do Sistema	001
Levantamento e Análise de Requisitos	002
Modelação Conceptual	003
Modelação Lógica	004
Implementação Física	005
Conclusão	006

001

DEFINIÇÃO DO SISTEMA

CONTEXTO INSTITUCIONAL

O CeSIUM, núcleo de estudantes de Engenharia Informática da Universidade do Minho, apoia e representa a comunidade académica. Organizam viagens e eventos que fortalecem o espírito de equipa e criam memórias únicas, complementando o percurso académico.

PROBLEMA + IDEIA

A história das viagens e iniciativas do CeSIUM está dispersa e em risco de esquecimento. Com o apoio dos Alumni, o projeto implementará um Sistema de Base de Dados centralizado para organizar, preservar e consultar a memória histórica do núcleo.

OBJETIVOS DO PROJETO

PRESERVAÇÃO DO LEGADO

Consolidação da memória histórica e cultural do CeSIUM num portfólio público centralizado.

GESTÃO EFICIENTE

Otimização do planejamento financeiro e apoio à tomada de decisão em futuros eventos e viagens.

SUCESSO ESTRATÉGICO

Maximização dos objetivos das viagens através da seleção informada e fundamentada de destinos.

SOBERANIA DOS DADOS

Garantia total da segurança, integridade e propriedade da informação institucional, evitando dependência de plataformas externas.

CONEXÃO INTERGERACIONAL

Fortalecimento do networking e partilha de conhecimento entre atuais e antigos estudantes.

ANÁLISE DA VIABILIDADE

VIABILIDADE TÉCNICA

A existência de **recursos humanos** qualificados no CeSIUM, nomeadamente estudantes e *alumni* de Engenharia Informática, assegura a capacidade de desenvolvimento e manutenção do sistema com recurso a tecnologias contemporâneas.

VIABILIDADE ECONÓMICA

○ **baixo investimento inicial**, graças ao desenvolvimento interno pela equipa do CAOS, e os custos restritos à manutenção garantem a sustentabilidade do modelo. O apoio financeiro dos *alumni* e a perspectiva de novos patrocínios reforçam a sua viabilidade.

VIABILIDADE OPERACIONAL

A previsão de **adesão natural da comunidade**, impulsionada pelo interesse na partilha de experiências e registo de viagens, valida a iniciativa.

RECURSOS HUMANOS

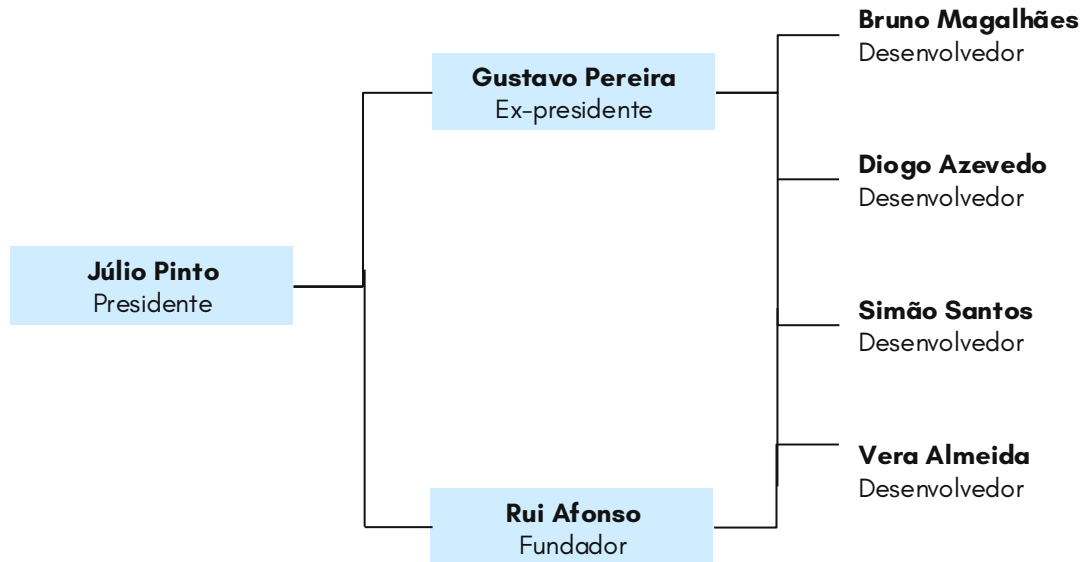


DIAGRAMA DE GANTT

O corpo colaborativo e a liderança do núcleo elaboraram um plano de trabalhos em diagrama de Gantt, definindo o ciclo de vida do sistema, as tarefas e os intervenientes.

A calendarização **atribui maior duração às etapas críticas de modelação e implementação técnica**, com margens de segurança para contingências ou flutuações na disponibilidade da equipa.

Implementação do sistema de base de dados
BeLIUM Vagens

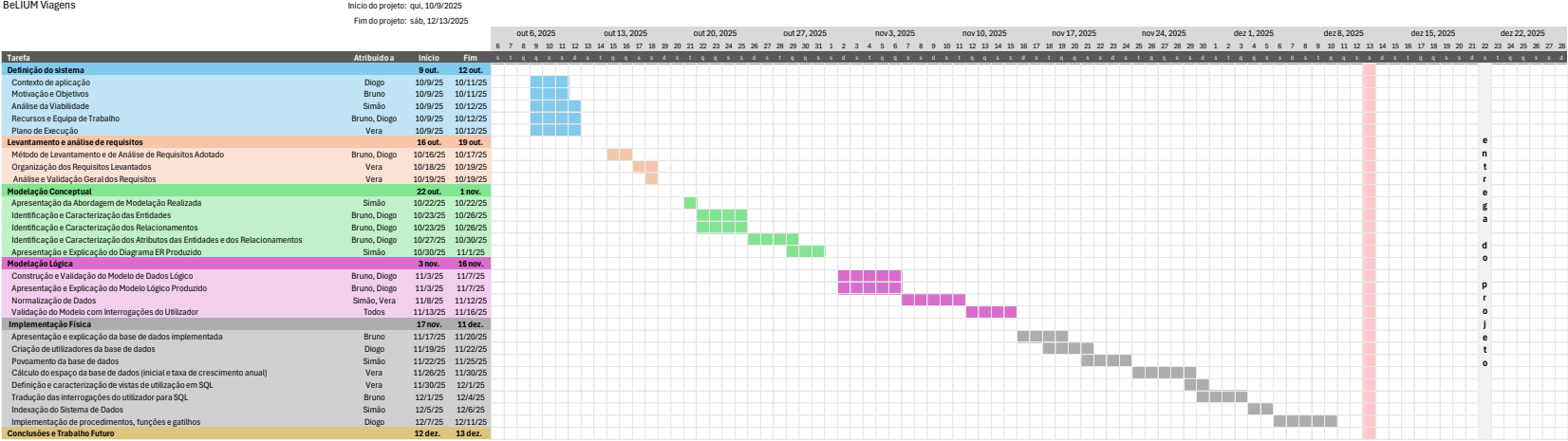


Fig 1. Diagrama de Gantt

002

LEVANTAMENTO E ANÁLISE DE REQUISITOS

MÉTODO DE LEVAMENTO DE REQUISITOS

O levantamento de requisitos para o projeto foi conduzido através de uma metodologia mista, que **integrou reuniões internas e inquéritos à comunidade académica**, com o intuito de alinhar o sistema às expectativas dos alumni e do CeSIUM.

ORGANIZAÇÃO DOS REQUISITOS LEVANTADOS

Com o intuito de sistematizar as especificações da base de dados, a equipa optou por **classificar os requisitos em três categorias distintas**, alinhadas com a natureza das operações a realizar.

12

(Definição e criação da estrutura da Base de Dados)

20

(Inserção e gestão da informação)

12

(Gestão das permissões de acesso por parte dos diferentes utilizadores)

Número	Data	Hora	Descrição	Vista de Utilização	Fonte	Analista
RD1	20/10/2025	10:16	Uma viagem é identificada pelo seu ID. Cada viagem contém um objetivo, uma data inicial e uma data final, um custo associado, o número de participantes, o número de paragens e um ou mais tipos de deslocamento.	Utilizadores	Reunião do CCP	Desenvolvedores
RD2	20/10/2025	10:18	Um sócio pode publicar viagens. Cada viagem pode ou não ser publicada por um ou mais sócios.	Sócios	Reunião do CCP	Desenvolvedores
RD3	20/10/2025	10:20	Cada sócio é caracterizado pelo seu número de sócio e, opcionalmente, pelo estatuto dentro do núcleo e por uma foto de perfil.	Sócios	Reunião do CCP	Desenvolvedores
RD4	20/10/2025	10:32	Uma viagem possui pelo menos uma paragem. Cada paragem pertence obrigatoriamente a uma única viagem. A viagem deve, também, possuir o número de paragens associadas.	Utilizadores	Reunião do CCP	Desenvolvedores

Fig 2. Excerto da tabela de requisitos de descrição

Número	Data	Hora	Descrição	Vista de Utilização	Fonte	Analista
RM1	27/10/2025	14:10	Registar novos sócios e alterar os dados dos existentes.	Administração	Reunião do CCP	Desenvolvedores
RM2	27/10/2025	14:12	Registar novos utilizadores e alterar os dados dos existentes.	Administração	Reunião do CCP	Desenvolvedores
RM3	27/10/2025	14:16	Listar, por ordem, todas as paragens de uma dada viagem.	Utilizadores/ Administração	Reunião do CCP	Desenvolvedores
RM4	27/10/2025	14:20	Listar todos os sócios (número de sócio, nome e estatuto) que realizaram uma determinada viagem (por ordem alfabética).	Utilizadores	Reunião do CCP	Desenvolvedores

Fig 3. Excerto da tabela de requisitos de manipulação

Número	Data	Hora	Descrição	Vista de utilização	Fonte	Analista
RC1	28/10/2025	14:39	Para se registar, um utilizador deve pedir diretamente a um administrador ou enviar um pedido, por email. Se o pedido for aceite, um administrador deve registá-lo no sistema e comunicá-lo do seu ID e da sua password.	Administração	Reunião do CCP	Desenvolvedores
RC2	28/10/2025	14:42	Um utilizador pode consultar todos os dados dos sócios, das viagens e das paragens no sistema.	Utilizadores	Reunião do CCP	Desenvolvedores
RC3	28/10/2025	14:46	Um utilizador pode atualizar a sua password, se souber a atual.	Utilizadores	Reunião do CCP	Desenvolvedores
RC4	28/10/2025	14:54	Não devem existir múltiplos registos de utilizadores com o mesmo email.	Utilizadores	Reunião do CCP	Desenvolvedores

Fig 4. Excerto da tabela de requisitos de controlo

ANÁLISE E VALIDAÇÃO GERAL DOS REQUISITOS

A equipa técnica e os supervisores procederam à **revisão e correção das vistas, vertentes e requisitos**.

Aprovado unanimemente, o documento estabelece uma base sólida para as fases subsequentes do desenvolvimento do projeto.

003

MODELAÇÃO CONCEPTUAL

APRESENTAÇÃO DA ABORDAGEM DE MODELAÇÃO REALIZADA

Após a análise de requisitos, a equipa estruturou a base de dados com um **diagrama Entidade-Relacionamento** (ER) no brModelo, usando a notação **Peter Chen Min-Max**.

Definiu-se **entidades, relacionamentos, atributos e cardinalidades**, com uma abordagem monovista para coerência e simplicidade, dado o volume de requisitos.

ENTIDADES

Entidade	Definição	Requisito
Utilizador	Entidade genérica que representa todos os indivíduos que interagem com o sistema.	RD7
Sócio	Entidade derivada de 'Utilizador'. Corresponde a um sócio da organização.	RD3
Viagem	Atividade principal do sistema, correspondente às viagens publicadas pelos sócios e visualizadas pelos utilizadores.	RD1
Paragem	Ponto incluído no itinerário de uma viagem. Depende de uma viagem para existir e ser identificado.	RD5
Patrocinador	Organização ou instituição que apoia financeiramente determinadas viagens, possuindo existência independente.	RD9

Fig 5. Entidades identificadas

RELACIONAMENTOS

Entidade	Relacionamento	Cardinalidade	Participação	Entidade	Requisito
Viagem	Possui	1 : N	T : T	Paragem	RD4
Viagem	Patrocinada	N : M	P : P	Patrocinador	RD8
Sócio	Publica	N : M	P : T	Viagem	RD2
Utilizador	Visualiza	N : M	P : P	Viagem	RD12
Sócio	-	1 : 1	T : P	Utilizador	RD6

Fig 6. Relacionamentos identificados

ATRIBUTOS

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
ID	Simples	INT	Não	Identificador único do patrocinador	1
Nome	Simples	VARCHAR (40)	Não	Nome do patrocinador	Yari Labs

Fig 7. Atributos identificados acerca da entidade **Patrocinador**

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
Nr. Sócio	Simples	INT	Não	Número de sócio no CeSIUM	2468
Estatuto	Simples	ENUM (...)	Sim	Define o estatuto do sócio no CeSIUM	Alumni
Foto_Perfil	Simples	VARCHAR (100)	Sim	Caminho para a fotografia de perfil do sócio	/imagens/FP1854.png

Fig 8. Atributos identificados acerca da entidade **Sócio**

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
ID	Simples	INT	Não	Identificador único do utilizador	2
Nome	Simples	VARCHAR (100)	Não	Nome e apelido do utilizador	Rui Almeida
Telemóvel	Simples	VARCHAR (13)	Não	Número de telemóvel do utilizador	+351911111111
Email	Simples	VARCHAR (100)	Não	Email do utilizador	ru1@email.com
Pass_Word	Simples	VARCHAR (100)	Não	Password do utilizador	ru123

Fig 9. Atributos identificados acerca da entidade **Utilizador**

ATRIBUTOS

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
ID	Simple	INT	Não	Identificador único da viagem	10
Data_Início	Simple	DATETIME	Não	Data em que a viagem teve início	2023-10-22 19:15:00
Data_Término	Simple	DATETIME	Não	Data em que a viagem terminou	2023-10-25 19:17:00
Objetivo	Simple	ENUM (...)	Não	Objetivo da viagem	Pedagógico
Custo	Simple	DECIMAL (8,2)	Não	Custo da viagem	23.50
Nr_Participantes	Simple	INT	Não	Número de participantes da viagem	6
Nr_Paragens	Simple	INT	Não	Número de paragens da viagem	3
Tipo_Deslocamento	Multivalor	ENUM (...)	Não	Tipos de deslocamento	Carro, Avião

Fig 10. Atributos identificados acerca da entidade Viagem

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
Nr	Simple	INT	Não	Identificador da paragem	45
Localização	Composto	-	-	Local onde se situa a paragem	-
País	Simple	VARCHAR (50)	Não	País onde se situa a paragem	Austria
Cidade	Simple	VARCHAR (50)	Não	Cidade onde se situa a paragem	Viena
Foto	Multivalor	VARCHAR (100)	Sim	Caminho para as fotos associadas à paragem	/Fotos/Viagem1_Vigo_01.png
Data_Chegada	Simple	DATETIME	Não	Data de chegada à paragem	2023-10-23 19:19:00
Data_Partida	Simple	DATETIME	Não	Data de partida da paragem	2023-10-24 19:21:00

Fig 11. Atributos identificados acerca da entidade Paragem

ATRIBUTOS

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
Detalhes	Composto	-	-	Detalhes do patrocínio	-
Valor	Simple	DECIMAL (8,2)	Não	Investimento do patrocinador na viagem	1200.05
Motivo	Multivalor	VARCHAR (200)	Não	Motivos para o patrocínio da viagem	Apoiar um projeto de investigação

Fig 12. Atributos identificados acerca do relacionamento Patrocinada

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
Descrição	Composto	-	-	Descrição da viagem	-
Avaliação	Simple	ENUM (...)	Não	Avaliação do sócio à viagem	Muito positiva
Comentário	Simple	VARCHAR (150)	Sim	Comentário do sócio à viagem	Expandiu os meus horizontes

Fig 13. Atributos identificados acerca do relacionamento Publica

Atributo	Tipo	Domínio	Nulo	Descrição	Exemplo
Reação	Simple	ENUM (...)	Não	Opinião do utilizador em relação à viagem	Negativa

Fig 14. Atributos identificados acerca do relacionamento Visualiza

APRESENTAÇÃO E EXPLICAÇÃO DO **MODELO ER** PRODUZIDO

A construção do diagrama obedeceu a um processo sequencial de cinco etapas, iniciando-se com a **identificação e inserção das entidades**, seguida do **estabelecimento dos relacionamentos**, **cardinalidades** e **participações**.

Posteriormente, procedeu-se à **caracterização detalhada dos atributos**, definindo **domínios**, **identificadores** e **opcionalidades**, e à validação do modelo face aos requisitos previamente levantados.

ESQUEMA CONCEPTUAL

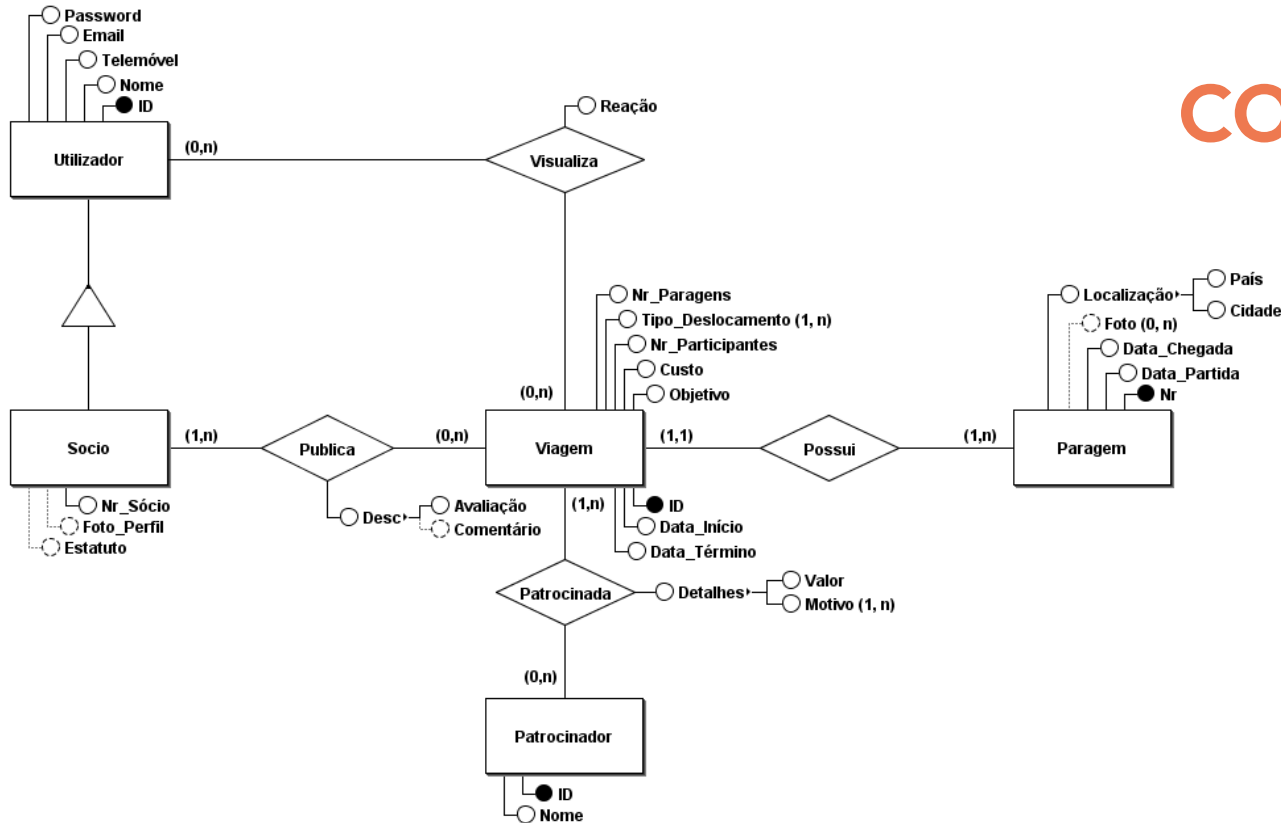


Fig 15. Esquema conceptual final

004

MODELAÇÃO LÓGICA

ANÁLISE E VALIDAÇÃO GERAL DOS REQUISITOS

A transição para a modelação lógica, visando a implementação em MySQL, foi efetuada através do MySQL Workbench com uma abordagem monovista.

As **entidades foram convertidas em tabelas relacionais, desmembrando atributos compostos e autonomizando atributos multivalor.**

Os relacionamentos foram mapeados de acordo com as suas cardinalidades.

As relações **muitos-para-muitos** resultaram na **criação de tabelas intermédias**, as relações **um-para-muitos** recorreram à **utilização de chaves estrangeiras**, e as relações de **especialização** foram **mantidas em tabelas distintas**, com a **chave primária da entidade especializada coincidindo com a da entidade genérica**, de forma a garantir a integridade referencial.

ESQUEMA LÓGICO

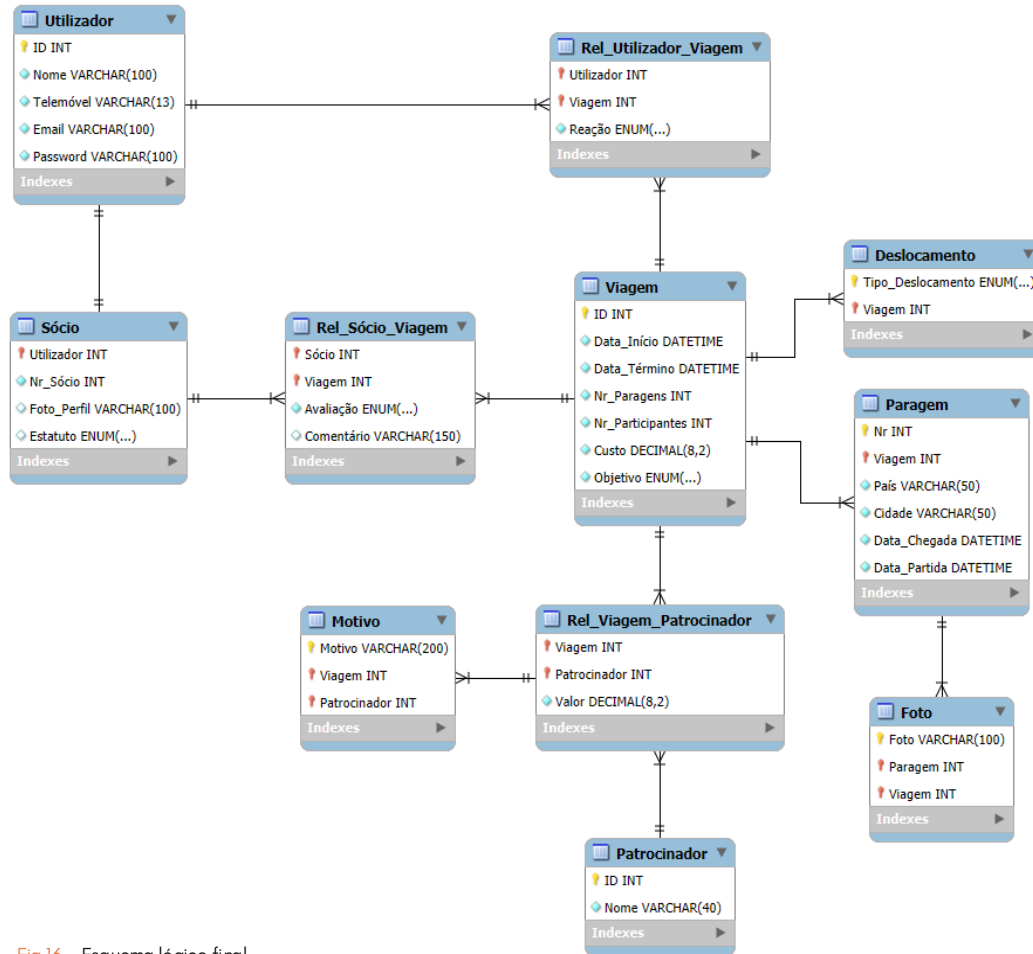


Fig 16. Esquema lógico final

DICIONÁRIO DE DADOS

Atributo	Domínio	Nulo	Chave Estrangeira
<u>ID</u>	INT	Não	Não
Nome	VARCHAR (100)	Não	Não
Telemóvel	VARCHAR (13)	Não	Não
Email	VARCHAR (100)	Não	Não
Password	VARCHAR (100)	Não	Não

Fig 17. Dicionário de dados da relação **Utilizador**

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Utilizador</u>	INT	Não	Sim
Nir_Sócio	INT	Não	Não
Foto_Perfil	VARCHAR (100)	Sim	Não
Estatuto	ENUM (...)	Sim	Não

Fig 18. Dicionário de dados da relação **Sócio**

Atributo	Domínio	Nulo	Chave Estrangeira
<u>ID</u>	INT	Não	Não
Nome	VARCHAR (40)	Não	Não

Fig 19. Dicionário de dados da relação **Patrocinador**

DICIONÁRIO DE DADOS

Atributo	Domínio	Nulo	Chave Estrangeira
<u>ID</u>	INT	Não	Não
Data_Início	DATETIME	Não	Não
Data_Término	DATETIME	Não	Não
Nr_Paragens	INT	Não	Não
Nr_Participantes	INT	Não	Não
Objetivo	ENUM (...)	Não	Não
Custo	DECIMAL (8,2)	Não	Não

Fig 20. Dicionário de dados da relação Viagem

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Nr</u>	INT	Não	Não
<u>Viagem</u>	INT	Não	Sim
País	VARCHAR (50)	Não	Não
Cidade	VARCHAR (50)	Não	Não
Data_Partida	DATETIME	Não	Não
Data_Chegada	DATETIME	Não	Não

Fig 21. Dicionário de dados da relação Paragem

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Utilizador</u>	INT	Não	Sim
<u>Viagem</u>	INT	Não	Sim
Reação	ENUM (...)	Não	Não

Fig 22. Dicionário de dados da relação Rel_Utilizador_Viagem

DICIONÁRIO DE DADOS

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Sócio</u>	INT	Não	Sim
<u>Viagem</u>	INT	Não	Sim
Avaliação	ENUM (...)	Não	Não
Comentário	VARCHAR (150)	Sim	Não

Fig 23. Dicionário de dados da relação Rel_Sócio_Viagem

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Patrocinador</u>	INT	Não	Sim
<u>Viagem</u>	INT	Não	Sim
Valor	DECIMAL (8,2)	Não	Não

Fig 24. Dicionário de dados da relação Rel_Patrocinador_Viagem

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Tipo_Deslocamento</u>	ENUM (...)	Não	Não
<u>Viagem</u>	INT	Não	Sim

Fig 25. Dicionário de dados da relação Deslocamento

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Foto</u>	VARCHAR (100)	Não	Não
<u>Paragem</u>	INT	Não	Sim
<u>Viagem</u>	INT	Não	Sim

Fig 26. Dicionário de dados da relação Foto

Atributo	Domínio	Nulo	Chave Estrangeira
<u>Motivo</u>	VARCHAR (200)	Não	Não
<u>Patrocinador</u>	INT	Não	Sim
<u>Viagem</u>	INT	Não	Sim

Fig 27. Dicionário de dados da relação Motivo

NORMALIZAÇÃO DE DADOS

A **normalização** é o processo sistemático de organização da base de dados, através da aplicação de regras lógicas designadas por **Formas Normais**.

O objetivo central é transformar estruturas de dados complexas em esquemas **mais simples, robustos e coerentes**. O modelo foi validado rigorosamente até à **Terceira Forma Normal** (3FN)

1ª FORMA NORMAL

Com o desígnio de **minimizar a redundância** e **eliminar anomalias de manipulação de dados**, aplicou-se a **Primeira Forma Normal** (1FN) para garantir a **atomicidade dos atributos** e a **inexistência de grupos repetidos**.

Este processo concretizou-se através da **decomposição de atributos compostos** e da **conversão de atributos multivalor em tabelas autónomas**, assegurando a integridade estrutural e a definição inequívoca das chaves primárias.

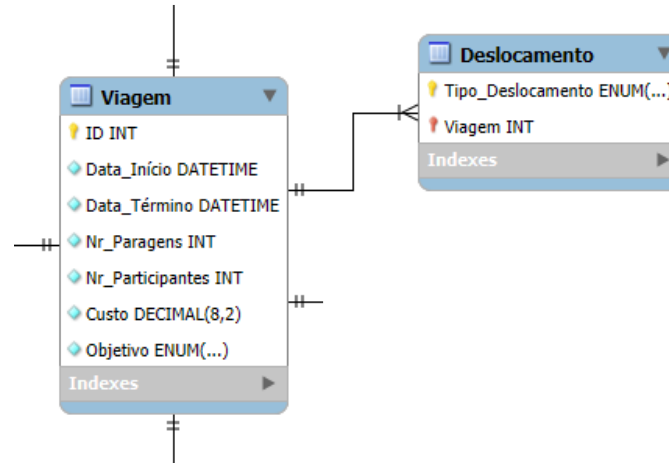


Fig 28. Relação Deslocamento

2ª FORMA NORMAL

A conformidade com a Segunda Forma Normal (2FN) foi assegurada pela garantia de que, cumulativamente ao cumprimento da 1FN, todos os **atributos não-chave apresentam dependência funcional total face à chave primária**.

Enquanto as tabelas de chave simples validam esta condição automaticamente, nas estruturas de chave composta, especificamente em 'Rel_Sócio_Viagem' e na entidade fraca Paragem, confirmou-se que os atributos dependem integralmente das respectivas combinações de identificadores, inexistindo quaisquer dependências parciais.

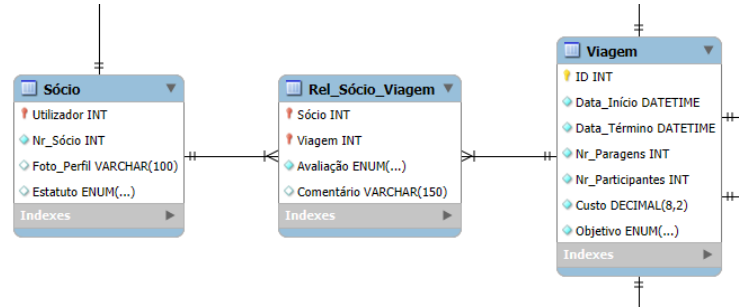


Fig 29. Relação Rel_Sócio_Viagem

3ª FORMA NORMAL

Utilizador	{ID} → Nome, Telemóvel, Email, Password
Sócio	{Utilizador} → Nr_Sócio, Foto_Perfil, Estatuto
Viagem	{ID} → Data_Início, Data_Término, Nr_Paragens, Nr_Participantes, Custo, Objetivo
Patrocinador	{ID} → Nome
Paragem	{Nr, Viagem} → Cidade, País, Data_Partida, Data_Chegada
Deslocamento	{Tipo_Deslocamento, Viagem}
Foto	{Foto, Paragem, Viagem}
Motivo	{Motivo, Viagem, Patrocinador}
Rel_Sócio_Viagem	{Sócio, Viagem} → Avaliação, Comentário
Rel_Utilizador_Viagem	{Utilizador, Viagem} → Reação
Rel_Viagem_Patrocinador	{Viagem, Patrocinador} → Valor

Fig 30. Dependências funcionais entre os diversos elementos de dados de cada relação

A Terceira Forma Normal (3NF) impõe, cumulativamente ao cumprimento da 2NF, a **inexistência de dependências transitivas entre atributos não-chave**. Neste âmbito, validou-se que a totalidade das tabelas satisfaz esta condição, garantindo-se assim a sua plena normalização até à 3NF.

VALIDAÇÃO DO MODELO COM INTERROGAÇÕES DO UTILIZADOR

A validação do modelo consistiu em **confrontar o esquema relacional com os requisitos** para garantir a **integridade** e a **normalização dos dados**.

O processo seguiu uma abordagem dual: primeiro, a **formalização das operações através da Álgebra Relacional** e, posteriormente, a **verificação prática na calculadora RelaX**.

ÁLGEBRA RELACIONAL

O requisito **RM3** consiste em **listar as paragens de uma viagem específica através da tabela 'Paragem'**.

O processo envolve a filtragem dos registos pelo **identificador da viagem fornecido** e a subsequente **ordenação por data de chegada**, de forma a apresentar a sequência correta do percurso.

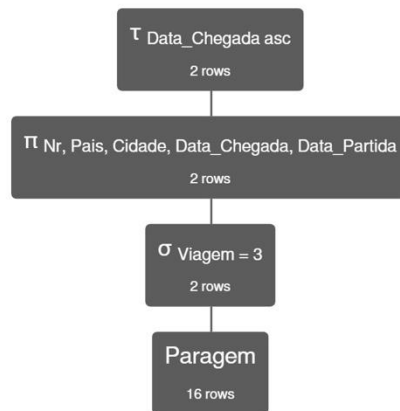


Fig 31. Árvore de interrogação do RM3

τ Data_Chegada ASC (π Nr, País, Cidade, Data_Chegada, Data_Partida(σ Viagem = ? Paragem))

ÁLGEBRA RELACIONAL

O requisito **RM4** visa **listar os sócios que participaram numa dada viagem**.

Para o concretizar, realiza-se a junção das tabelas 'Sócio', 'Rel_Sócio_Viagem' e 'Utilizador'. O processo finaliza com a filtragem pela viagem pretendida, a seleção dos campos necessários e a ordenação dos nomes.

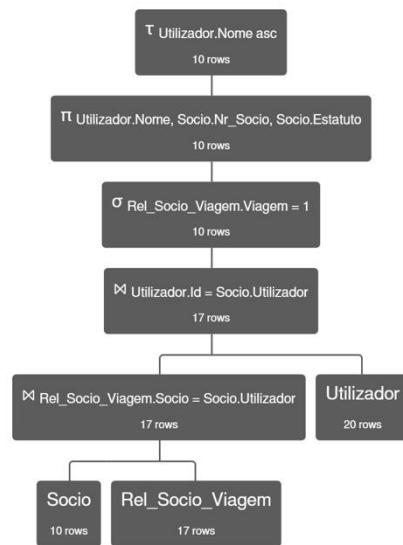


Fig 32. Árvore de interrogação do RM4

```
τ Nome ASC (π ID, Nome (σ Viagem = ? ((Sócio ⋈ Rel_Sócio_Viagem.Sócio = Socio.Utilizador Rel_Sócio_Viagem)
⋈ Utilizador.ID = Sócio.Utilizador Utilizador)))
```

ÁLGEBRA RELACIONAL

O requisito **RM17** quantifica a participação dos membros na plataforma.

O processo envolve a junção das tabelas 'Utilizador', 'Sócio' e, através de uma junção externa à esquerda, a tabela 'Rel_Sócio_Viagem'. Os dados são agrupados por sócio, contados e, finalmente, ordenado.

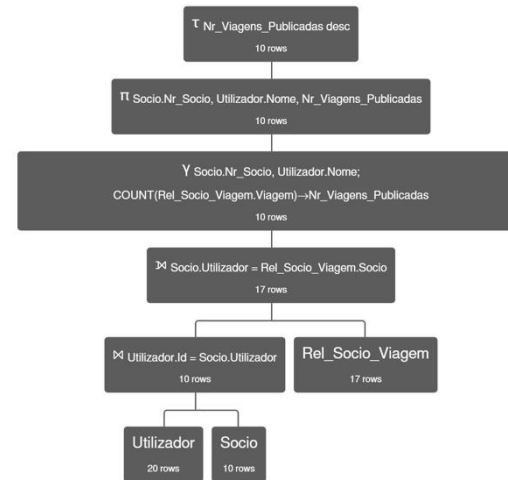


Fig 33. Árvore de interrogação do RM17

```

τ Nr_Viagens_Publicadas DESC (π Nr_Sócio, Nome, Nr_Viagens_Publicadas
    (γ Nr_Sócio, Nome, COUNT (Viagem) → Nr_Viagens_Publicadas
        ((Utilizador ⋈ Utilizador = ID Sócio) ⋈ Utilizador = Sócio Rel_Sócio_Viagem)))
    
```

ÁLGEBRA RELACIONAL

O requisito **RM20** visa **listar as avaliações e comentários dos sócios sobre uma viagem específica**.

O processo baseia-se na junção das tabelas 'Rel_Sócio_Viagem' e 'Sócio'. Após a filtragem pela viagem pretendida, o sistema extrai o número do sócio, a respetiva classificação e o comentário.

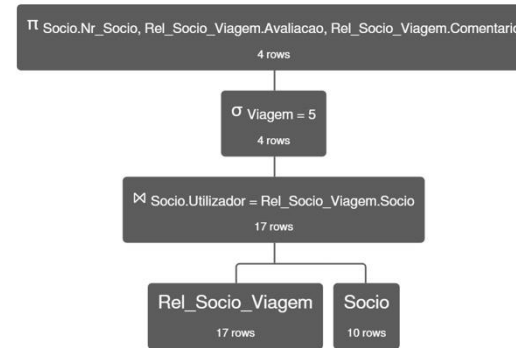


Fig 34. Árvore de interrogação do RM20

π Nr_Socio, Avaliação, Comentário (σ Viagem = ? (Rel_Socio_Viagem $\bowtie$ Utilizador = Sócio Sócio))

005

IMPLEMENTAÇÃO FÍSICA

INTRODUÇÃO À IMPLEMENTAÇÃO FÍSICA

Após a construção do modelo lógico, a **implementação física** da base de dados consistiu na **tradução sistemática desse modelo para um conjunto de instruções SQL** interpretadas pelo SGBD MySQL.

Este processo assegura que toda a estrutura de tabelas planeada anteriormente seja devidamente criada e configurada no sistema de gestão de base de dados para entrar em fase operacional

INTRODUÇÃO À IMPLEMENTAÇÃO FÍSICA

A fase de preparação do ambiente foca-se na execução de comandos iniciais que garantem a **criação e seleção correta da base de dados** 'BeLIUM_Viagens'.

A fase de preparação executa comandos iniciais para criar e seleccionar a base de dados 'BeLIUM_Viagens'.

As cláusulas `IF EXISTS` **e** `IF NOT EXISTS` **garantem a re-execução do código sem erros.**

O script **elimina e cria a base de dados, ativando-a em seguida com o comando** `USE`.

```
DROP DATABASE IF EXISTS  
    BeLIUM_Viagens;  
CREATE DATABASE IF NOT  
EXISTS BeLIUM_Viagens;  
    USE BeLIUM_Viagens;
```

ORDEM DE CRIAÇÃO E DEPENDÊNCIAS

Para garantir a integridade do modelo, é fundamental respeitar a hierarquia das **Chaves Estrangeiras (FK)**, assegurando que qualquer tabela referenciada seja criada antes da tabela que a referencia.

Assim, o script segue uma **sequência lógica** que começa pela definição das **Entidades Independentes** (como Utilizador, Viagem e Patrocinador).

Seguem-se as **Entidades Dependentes ou Especializadas** (como Sócio e Paragem) e, por fim, são estruturadas as **Tabelas de Relacionamento N:M** e os **Atributos Multivalor**, garantindo que todas as dependências estejam corretamente mapeadas desde o início.

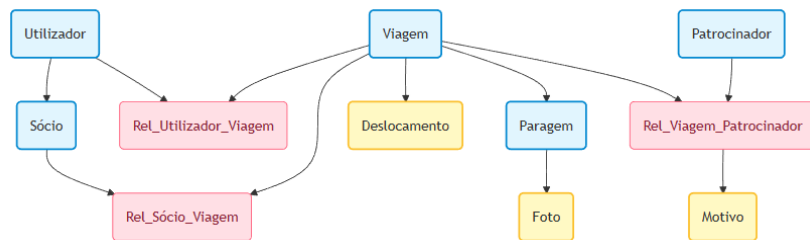


Fig 35 Grafo do diagrama de dependências da criação da BD

IMPLEMENTAÇÃO DE ENTIDADES INDEPENDENTES

Na tabela **Utilizador**, a gestão de identificadores é automatizada através do uso de `AUTO_INCREMENT`, enquanto a integridade dos dados é assegurada por restrições `CHECK`, que validam rigorosamente o formato do e-mail e do telemóvel.

Para a especialização de **Sócio**, a arquitetura define que a sua **Chave Primária** atua simultaneamente como **Chave Estrangeira**, estabelecendo uma ligação direta à tabela Utilizador.

Adicionalmente, o controlo de domínios para o estatuto dos membros é garantido pelo tipo `ENUM`, que restringe as entradas aos valores predefinidos 'Direção' e 'Alumni'.

CRIAÇÃO DAS TABELAS

```
CREATE TABLE IF NOT EXISTS Sócio (  
  
  Utilizador INT UNSIGNED NOT NULL,  
  Nr_Sócio INT UNSIGNED NOT NULL UNIQUE,  
  Foto_Perfil VARCHAR(100) NULL,  
  Estatuto ENUM('Direção','Alumni') NULL,  
  
  PRIMARY KEY (Utilizador),  
  
  CONSTRAINT FK_Sócio_To_Utilizador FOREIGN  
  KEY (Utilizador) REFERENCES Utilizador(ID),  
  
  CHECK (Foto_Perfil IS NULL OR Foto_Perfil  
  LIKE 'Imagens/%.png')  
);
```

```
CREATE TABLE IF NOT EXISTS Utilizador (  
  
  ID INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  Nome VARCHAR(100) NOT NULL,  
  Telemóvel VARCHAR(13) NOT NULL,  
  Email VARCHAR(100) NOT NULL UNIQUE,  
  Pass_Word VARCHAR(100) NOT NULL,  
  
  PRIMARY KEY (ID),  
  
  CHECK (Telemóvel LIKE '+%'),  
  CHECK (Email LIKE '%@%.%')  
);
```

RELACIONAMENTOS E INTERAÇÕES

A arquitetura de relações do sistema é composta por três tabelas fundamentais que cruzam dados de diferentes entidades.

A tabela **Rel_Utilizador_Viagem** gere as reações binárias através de `ENUM`, enquanto a tabela **Rel_Sócio_Viagem** expande esta capacidade para os sócios, suportando métricas de avaliação e observações textuais.

Complementarmente, a tabela de ligação **Rel_Viagem_Patrocinador** é responsável por mapear o suporte financeiro, vinculando os montantes dos patrocinadores a cada viagem, garantindo assim a rastreabilidade total dos patrocínios no modelo de dados.

CRIAÇÃO DAS TABELAS

```
CREATE      TABLE      IF      NOT      EXISTS

Rel_Utilizador_Viagem (

Utilizador INT UNSIGNED NOT NULL,

Viagem INT UNSIGNED NOT NULL,

Reação ENUM('Negativa','Positiva') NOT NULL,

PRIMARY KEY (Utilizador,Viagem),

CONSTRAINT

FK_Rel_Utilizador_Viagem_To_Utilizador FOREIGN

KEY (Utilizador) REFERENCES Utilizador(ID),

CONSTRAINT FK_Rel_Utilizador_Viagem_To_Viagem

FOREIGN KEY (Viagem) REFERENCES Viagem(ID)

);
```

IDENTIFICAÇÃO DOS UTILIZADORES

A base de dados usa uma arquitetura de segurança com três níveis de acesso.

Na base, encontram-se os **Utilizadores**, que dispõem de permissões restritas, vocacionadas exclusivamente para a consulta de informações relativas às viagens.

Num nível intermédio situam-se os **Sócios**, aos quais compete a gestão operacional das viagens e dos dados a elas associados.

Por fim, o topo da hierarquia é ocupado pelos **Administradores**, que exercem controlo integral e irrestrito sobre todas as funcionalidades do sistema.

Perfil	Descrição das Permissões	Restrições Principais
Utilizadores	Consultar dados centrais, alterar a própria password e gerir as suas próprias visualizações de viagens.	Nível de acesso mínimo e restrito.
Sócios	Herda as permissões de Utilizador. Pode inserir viagens, paragens, transportes e fotos. Consulta total, exceto dados de outros utilizadores.	Bloqueio total de leitura/escrita na tabela Utilizador.
Administradores	Controlo total. Gerem utilizadores, sócios, patrocínios e a estrutura da base de dados. Responsáveis pelo registo de novos acessos.	Reservado inicialmente aos membros do CCP.

Fig 36 Descrição da **hierarquia** do sistema BeLIUM Viagens

CRIAÇÃO DOS UTILIZADORES

```
CREATE USER IF NOT EXISTS  
'UTILIZADOR'@'localhost'  
IDENTIFIED BY  
'BeLIUM_Viagens_UTILIZADOR_1025';
```

```
CREATE USER IF NOT EXISTS  
'SOCIO'@'localhost'  
IDENTIFIED BY  
'BeLIUM_Viagens_SOCIO_2047';
```

```
CREATE USER IF NOT EXISTS  
'ADMIN'@'localhost'  
IDENTIFIED BY 'BeLIUM_Viagens_VDBS';
```

ESTABELECIMENTO DE PERMISSÕES DOS UTILIZADORES

```
GRANT SELECT ON BeLIUM_Viagens.Sócio TO 'UTILIZADOR'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Viagem TO 'UTILIZADOR'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Paragem TO 'UTILIZADOR'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Deslocamento TO 'UTILIZADOR'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Foto TO 'UTILIZADOR'@'localhost';  
GRANT EXECUTE ON PROCEDURE MudarPassword TO 'UTILIZADOR'@'localhost';  
GRANT EXECUTE ON PROCEDURE InserirVisualização TO 'UTILIZADOR'@'localhost';  
GRANT EXECUTE ON PROCEDURE AtualizarVisualização TO 'UTILIZADOR'@'localhost';  
GRANT EXECUTE ON PROCEDURE RemoverVisualização TO 'UTILIZADOR'@'localhost';
```

ESTABELECIMENTO DE PERMISSÕES DOS SÓCIOS

```
GRANT INSERT, UPDATE, DELETE ON BeLIUM_Viagens.Viagem TO 'SOCIO'@'localhost';  
GRANT INSERT, UPDATE, DELETE ON BeLIUM_Viagens.Paragem TO 'SOCIO'@'localhost';  
GRANT INSERT, UPDATE, DELETE ON BeLIUM_Viagens.Deslocamento TO 'SOCIO'@'localhost';  
GRANT INSERT, UPDATE, DELETE ON BeLIUM_Viagens.Foto TO 'SOCIO'@'localhost';  
GRANT INSERT, UPDATE, DELETE ON BeLIUM_Viagens.Rel_Sócio_Viagem TO 'SOCIO'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Rel_Utilizador_Viagem TO 'SOCIO'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Rel_Sócio_Viagem TO 'SOCIO'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Rel_Viagem_Patrocinador TO 'SOCIO'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Patrocinador TO 'SOCIO'@'localhost';  
GRANT SELECT ON BeLIUM_Viagens.Motivo TO 'SOCIO'@'localhost';
```

ESTABELECIMENTO DE PERMISSÕES DOS ADMINISTRADORES

```
GRANT ALL PRIVILEGES ON BeLIUM_Viagens.TO 'ADMIN'@'localhost' WITH GRANT OPTION;
```

PRINCÍPIOS PARA O POVOAMENTO DA BASE DE DADOS

O povoamento inicial da base de dados é realizado através de **stored procedures** e instruções **INSERT**, com o objetivo de validar os requisitos funcionais do sistema.

O aspeto mais crítico deste processo é o cumprimento da **integridade referencial**, o que exige que os registos sejam inseridos seguindo uma **ordem topológica** rigorosa.

Esta sequência começa pelas entidades independentes, como **Utilizador**, **Viagem** e **Patrocinador**, seguida pelas inserções nas tabelas dependentes: primeiro as extensões diretas e, finalmente, as tabelas de relacionamento. Esta estratégia garante que nenhuma chave estrangeira tente referenciar um registo que ainda não existe no sistema.

ESTRATÉGIA ADOTADA PARA O POVOAMENTO DA BASE DE DADOS

A utilização de **procedimentos armazenados** para o povoamento e gestão do sistema 'BeLIUM Viagens' baseia-se num modelo de eficiência e segurança. Em vez de executar comandos SQL isolados, a lógica de inserção é encapsulada no servidor, garantindo quatro benefícios fundamentais: persistência, reutilização, manutenção e segurança.

POVOAMENTO DA BASE DE DADOS

Relação	Número de registos	Método de obtenção dos dados
Utilizador	20	Selecionaram-se 10 sócios da organização (incluindo os membros do CCP) e 10 voluntários não sócios.
Viagem	5	Selecionou-se uma viagem realizada por cada desenvolvedor e uma viagem coletiva da organização.
Patrocinador	3	Selecionou-se o patrocinador da viagem coletiva e dois patrocinadores históricos do CeSIUM.

Fig 37 Metodologia de obtenção de dados para o povoamento das relações de entidades independentes

POVOAMENTO DA BASE DE DADOS

ID	Nome	Telemóvel	Email
1	Bruno Magalhães	+351937426819	brunomagalhaes_cesium_1728@gmail.com
2	Vera Almeida	+351963817402	veraalmeida_cesium_1854@gmail.com
3	Diogo Azevedo	+351912345678	diogoazevedo_cesium_1912@gmail.com
4	Simão Santos	+351934781256	simaosantos_cesium_2037@gmail.com
5	Júlio Pinto	+351961482739	juliopinto_cesium_2149@gmail.com
6	Gustavo Pereira	+351926573814	gustavopereira_cesium_2263@gmail.com
7	Rui Afonso	+351968240157	ruiafonso_cesium_2378@gmail.com
8	Mário Rodrigues	+351913857462	mariorodrigues_cesium_2441@gmail.com
9	Humberto Gomes	+351947162805	humbertogomes_cesium_1986@gmail.com
10	Gonçalo Pombo	+351965028417	goncalopombo_cesium_2215@gmail.com
11	João Silva	+351912345679	joao.silva@email.com
12	Maria Santos	+351934781257	maria.santos@email.com
13	Pedro Almeida	+351961482740	pedro.almeida@email.com
14	Ana Costa	+351926573815	ana.costa@email.com
15	Rui Ferreira	+351968240158	rui.ferreira@email.com
16	Sofia Martins	+351913857463	sofia.martins@email.com
17	Tiago Lopes	+351947162806	tiago.lopes@email.com
18	Inês Rocha	+351965028418	ines.rocha@email.com
19	Carlos Pereira	+351932740681	carlos.pereira@email.com
20	Beatriz Nunes	+351951864237	beatriz.nunes@email.com

Fig 38 Povoamento realizado na relação Utilizador

ANÁLISE DA VIABILIDADE DE VISTAS

O uso de **Vistas** no sistema 'BeLIUM Viagens' funciona como uma camada de **tabelas virtuais** que simplifica o acesso aos dados e reforça a segurança. Ao contrário das tabelas físicas, as vistas não duplicam os dados; em vez disso, executam uma consulta SQL dinâmica sempre que são consultadas.

Esta abordagem é estrategicamente adotada por dois motivos principais: primeiro, pela **segurança**, uma vez que permite filtrar colunas ou registros sensíveis sem conceder acesso direto às tabelas base; segundo, pela **simplicificação**, pois encapsula operações complexas, como junções e agregações, numa interface mais intuitiva para o utilizador.

Embora apresentem a limitação técnica de não aceitarem **parâmetros de entrada** — o que impede a sua utilização em filtros dinâmicos em tempo de execução —, as vistas são a solução ideal para os requisitos **RM11, RM17 e RM19**.

IMPLEMENTAÇÃO DE VISTAS

```
CREATE VIEW VisualizarInvestimentoTotal AS
SELECT P.ID, P.Nome, SUM(RVP.Valor) AS Investimento_Total FROM Patrocinador AS P
      LEFT OUTER JOIN Rel_Viagem_Patrocinador AS RVP ON P.ID = RVP.Patrocinador
      GROUP BY P.ID, P.Nome;
```

```
CREATE VIEW VisualizarNrPublicações AS
SELECT S.Nr_Sócio, U.Nome, COUNT(RSV.Viagem) AS Nr_Viagens_Publicadas FROM Utilizador AS U
      INNER JOIN Sócio AS S ON U.ID = S.Utilizador
      LEFT OUTER JOIN Rel_Sócio_Viagem AS RSV ON S.Utilizador = RSV.Sócio
      GROUP BY S.Nr_Sócio, U.Nome
      ORDER BY Nr_Viagens_Publicadas DESC;
```

METODOLOGIA PARA A SATISFAÇÃO DAS INTERROGAÇÕES DO UTILIZADOR

A **transição da álgebra relacional para o SQL** no sistema 'BeLIUM Viagens' é apresentada como um processo de adaptação sintática que mantém a semântica das operações originais, mas tira proveito da flexibilidade técnica do SQL. Ao contrário da ferramenta Relax, o SQL permite a parametrização de consultas, o que possibilita a preparação de instruções cujos valores são definidos apenas no momento da execução.

Para **otimizar** o desempenho e a organização do sistema, a escolha do método de implementação de cada interrogação baseou-se na sua complexidade e necessidade de variáveis: as **instruções preparadas** foram selecionadas para consultas simples com poucos parâmetros; as **vistas** foram aplicadas a interrogações estáticas que não requerem parâmetros; e os **procedimentos armazenados** foram reservados para as operações de maior complexidade.

RESOLUÇÃO DAS INTERROGAÇÕES DO UTILIZADOR

- Requisito **RM3** implementado através de uma instrução preparada

```
PREPARE RM3 FROM  
'SELECT Nr, País, Cidade, Data_Chegada, Data_Partida FROM Paragem  
    WHERE Viagem = ?  
    ORDER BY Data_Chegada ASC;'  
SET @ViagemRM3 = 1;  
EXECUTE RM3 USING @ViagemRM3;  
DEALLOCATE PREPARE RM3;
```

RESOLUÇÃO DAS INTERROGAÇÕES DO UTILIZADOR

- Requisito **RM9** implementado através de um procedimento

```
SELECT * FROM Viagem
WHERE (? IS NULL OR Data_Início >= ?) AND
      (? IS NULL OR Data_Término <= ?) AND
      (? IS NULL OR Custo >= ?) AND
      (? IS NULL OR Custo <= ?) AND
      (? IS NULL OR Objetivo = ?);
```

RESOLUÇÃO DAS INTERROGAÇÕES DO UTILIZADOR

- Requisito **RM11** implementado através de uma vista e com o auxílio de uma função

```
SELECT P.ID, P.Nome, CalcularInvestimentoTotal(P.ID) AS Investimento_Total
FROM Patrocinador AS P
LEFT OUTER JOIN Rel_Viagem_Patrocinador AS RVP ON P.ID = RVP.Patrocinador
GROUP BY P.ID, P.Nome;
```

RESOLUÇÃO DAS INTERROGAÇÕES DO UTILIZADOR

- Requisito **RM19** implementado através de uma vista

```
SELECT V.*, COUNT(RUV.Viagem) AS Nr_Visualizações FROM Viagem AS V
      LEFT OUTER JOIN Rel_Utilizador_Viagem AS RUV ON V.ID = RUV.Viagem
      GROUP BY V.ID
      ORDER BY Nr_Visualizações DESC;
```

ANÁLISE DA VIABILIDADE DE ÍNDICES

A utilização de índices no sistema 'BeLIUM Viagens' é encarada como uma ferramenta de otimização de desempenho, funcionando como estruturas de dados que aceleram a localização de registos sem percorrer toda a tabela. Esta funcionalidade é ideal para operações de filtragem e ordenação, como as previstas para a listagem de paragens e filtragem de viagens por localização ou datas.

Contudo, a equipa reconhece que os índices acarretam custos, nomeadamente o aumento do espaço de armazenamento e a redução da velocidade em operações de escrita. Atendendo ao volume reduzido de dados na fase inicial e ao facto de as consultas serem instantâneas, optou-se por não ativar índices de imediato. No entanto, foram já desenvolvidos e documentados scripts para a criação de índices para os atributos mais consultados.

IMPLEMENTAÇÃO DE ÍNDICES

```
CREATE INDEX IDX_Paragem_Data_Chegada  
    ON Paragem (Data_Chegada);  
CREATE INDEX IDX_Paragem_Localização  
    ON Paragem (País, Cidade);  
  
CREATE INDEX IDX_Viagem_Detalhes  
    ON Viagem (Data_Início, Data_Término, Custo, Objetivo);  
  
CREATE INDEX IDX_Rel_Utilizador_Viagem_Reação  
    ON Rel_Utilizador_Viagem (Reação);
```

ANÁLISE DA VIABILIDADE DE SQL AVANÇADO

O MySQL é utilizado como um ambiente de programação completo, permitindo o uso de **procedimentos armazenados**, **funções** e **gatilhos**. Esta abordagem permite que a lógica de negócio e as regras de validação sejam encapsuladas diretamente no servidor, o que reduz a complexidade da aplicação e centraliza o controlo de operações críticas.

A grande vantagem desta estratégia é **a execução no próprio servidor de base de dados**, o que minimiza o tráfego de rede e melhora o desempenho através da pré-compilação de código.

IMPLEMENTAÇÃO DE TRIGGERS

```
CREATE TRIGGER AtualizaNrParagens
AFTER INSERT ON Paragem FOR EACH ROW
BEGIN
    UPDATE Viagem
    SET Nr_Paragens = Nr_Paragens + 1
    WHERE ID = NEW.Viagem;
END
```

IMPLEMENTAÇÃO DE FUNCTIONS

```
CREATE FUNCTION CalcularInvestimentoTotal (ID_Patrocinador INT)
RETURNS DECIMAL(10,2) DETERMINISTIC
BEGIN
    RETURN (SELECT SUM(Valor) FROM Rel_Viagem_Patrocinador
            WHERE Patrocinador = ID_Patrocinador);
END
```

IMPLEMENTAÇÃO DE PROCEDURES

```
CREATE PROCEDURE InserirUtilizador (  
    IN p_Nome VARCHAR(100),  
    IN p_Telemovel VARCHAR(13),  
    IN p_Email VARCHAR(100),  
    IN p_Pass_Word VARCHAR(100)  
)  
BEGIN  
    INSERT INTO Utilizador (Nome, Telemóvel, Email, Pass_Word)  
        VALUES (p_Nome, p_Telemovel, p_Email, p_Pass_Word);  
END
```

IMPLEMENTAÇÃO DE PROCEDURES

```
CREATE PROCEDURE AutentificaUtilizador(  
    IN p_email VARCHAR(100),  
    IN p_password VARCHAR(100),  
    OUT p_ID INT,  
    OUT p_mensagem VARCHAR(100)  
)  
BEGIN  
    DECLARE v_ID INT;  
    DECLARE v_password VARCHAR(100);  
    SET p_ID = NULL;  
    SET p_mensagem = NULL;  
    SELECT ID, Pass_Word INTO v_ID, v_password FROM Utilizador  
        WHERE Email = p_email;  
    IF v_ID IS NULL THEN  
        SET p_mensagem = 'O email é inválido';  
    ELSEIF v_password != p_password THEN  
        SET p_mensagem = 'A password é inválida';  
    ELSE SET p_ID = v_ID;  
    END IF;  
END
```

BeLIUM Viagens

Do Minho para o Mundo